

Reproduction and Experimental Analysis of CP-LSH for Graph Representation Learning

Team Members

Bhavya Sri Kottana
Sri Shraddha Singirikonda
Bhavya Sri Mamidi

Department of Computer Science and Engineering
Indian Institute of Technology Bombay

May 2026

Outline

- 1 Introduction
- 2 Background
- 3 CP-LSH Framework
- 4 Methodology
- 5 Experimental Setup
- 6 Results
- 7 Conclusion

Graph Representation Learning

- Many real-world systems are naturally represented as graphs:
 - Citation networks
 - Social networks
 - Recommendation systems
- Goal: Learn low-dimensional node embeddings
- Applications:
 - Node classification
 - Link prediction
 - Clustering
 - Recommendation

Problem: Long-Tail Nodes

- Real-world graphs follow power-law degree distributions
- Majority of nodes have very few neighbors
- These sparse nodes are called: **Tail Nodes**
- Structure-only methods struggle because:
 - Tail nodes appear less in random walks
 - Receive fewer updates
 - Learn weak embeddings

Key Contributions

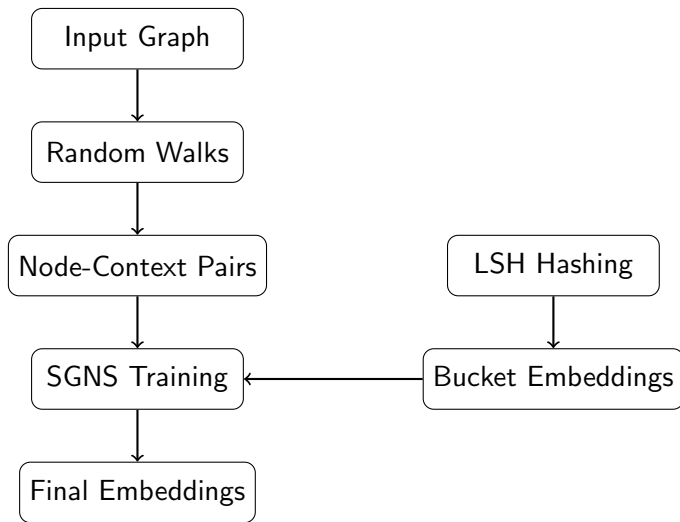
- ① Reproduced the CP-LSH framework
- ② Implemented:
 - DeepWalk baseline
 - LSH-based parameter sharing
 - SGNS optimization
- ③ Evaluated:
 - Node classification
 - Tail-node robustness
 - Runtime and memory
 - Embedding visualization
- ④ Compared observations with the original paper

- Foundational graph embedding method
- Pipeline:
 - 1 Generate random walks
 - 2 Treat walks as sentences
 - 3 Train Skip-Gram model
- Advantages:
 - Captures structural similarity
 - Simple and scalable
- Limitations:
 - Ignores node features
 - Large parameter count
 - Weak tail-node performance

Motivation Behind CP-LSH

- Improve representation quality for sparse nodes
- Combine:
 - Structural learning
 - Content similarity
- Instead of one embedding per node:
 - Hash similar nodes into shared buckets
 - Learn bucket embeddings
- Benefits:
 - Better scalability
 - Reduced redundancy
 - Improved information sharing

Overall Pipeline



$$h_u(p) = \text{Sgn}(u \cdot p)$$

- Random hyperplane based hashing
- Similar nodes collide into same buckets
- Shared buckets enable parameter sharing

Collision probability:

$$Pr[h(p) = h(q)] = 1 - \frac{\arccos(\Theta(p, q))}{\pi}$$

CP-LSH uses Skip-Gram with Negative Sampling (SGNS)

$$\log \sigma(s_u \cdot t_v) + k \mathbb{E}_{n \sim V} [\log \sigma(-s_u \cdot t_n)]$$

- Learn positive node-context pairs
- Push negative samples apart
- Preserve structural similarity

- Generate random walks from graph structure
- Extract node-context pairs using sliding window
- Apply LSH using:
 - Multiple hash tables
 - Random hyperplanes
- Nodes with similar features:
 - Share embeddings
 - Share training updates
- Optimize using SGNS

Dataset Statistics

Dataset	Nodes	Edges	Classes
Cora	2708	5429	7
Citeseer	3327	4732	6
DBLP	Large-scale	Dense	Multiple

- Citation network datasets
- Heavy-tailed degree distributions
- Suitable for tail-node evaluation

Experimental Setup

Baselines

- DeepWalk
- SVD
- DW+SVD
- CP-LSH

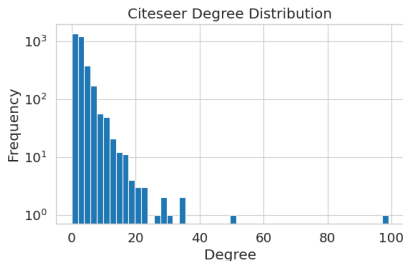
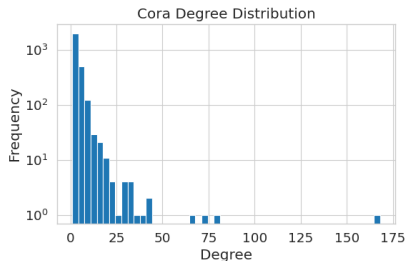
Implementation

- PyTorch
- Google Colab
- GPU acceleration

Evaluation Tasks

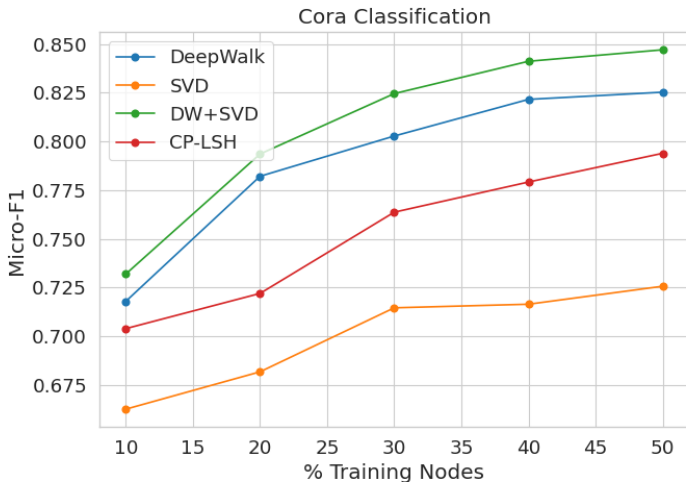
- Node Classification
- Link Prediction
- Tail-node Analysis
- Runtime and Memory

Degree Distribution



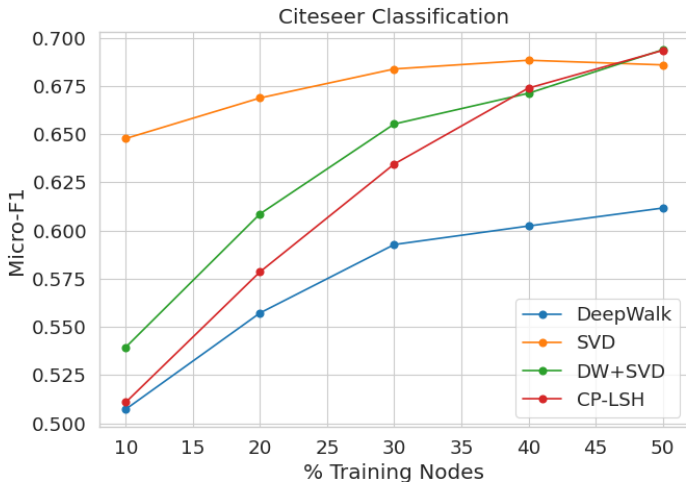
- Both datasets exhibit heavy-tailed distributions
- Majority of nodes are sparsely connected

Cora Classification Results



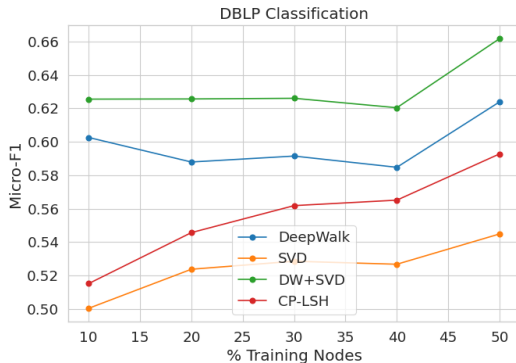
- CP-LSH remains competitive across training ratios
- Hybrid approaches achieve strongest performance

Citeseer Classification Results



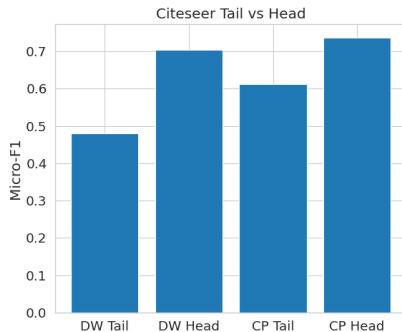
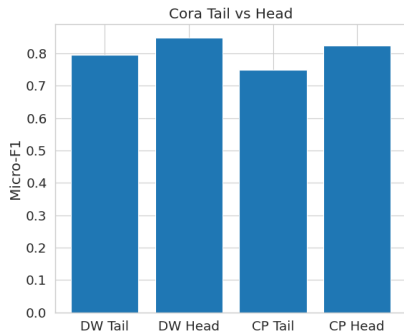
- CP-LSH improves with larger supervision
- Performance trends remain competitive

DBLP Classification Results



- CP-LSH scales well on larger graphs
- The original paper likely used stronger tuning and optimized training, giving better results for cp-lsh than our lightweight implementation.

Tail Node Analysis



- CP-LSH improves low-degree node performance
- Shared embeddings transfer semantic information

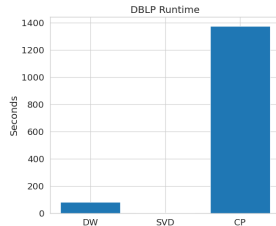
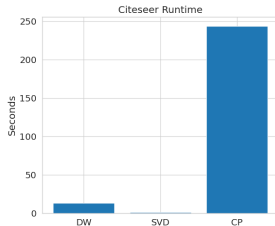
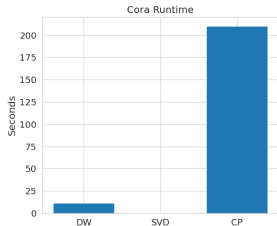
Link Prediction

- Evaluated using AUC score
- Task:
 - Predict missing links
 - Distinguish edges from random pairs
- Observations:
 - CP-LSH preserves structural similarity
 - Performs competitively with DeepWalk

Key Observation

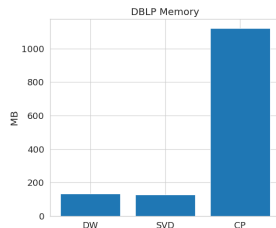
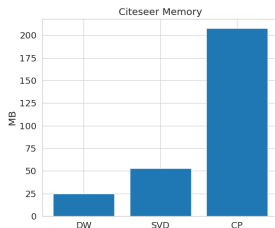
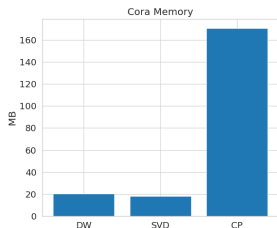
Integrating content information does not harm structural representation learning.

Runtime Analysis



- CP-LSH introduces hashing overhead
- Runtime increases due to bucket aggregation

Memory Analysis



- CP-LSH theoretically reduces parameter redundancy
- The practical implementation introduces hashing and embedding overhead, resulting in higher actual memory usage for smaller datasets

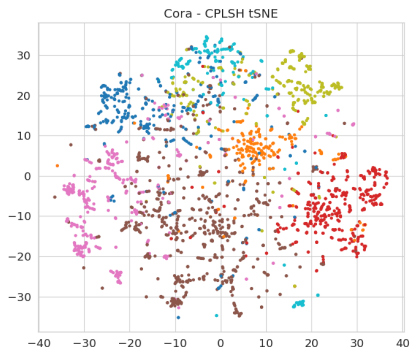
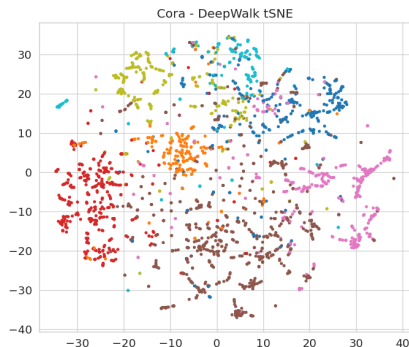
Sensitivity Analysis

- Studied effect of:
 - Number of hash tables
 - Number of hash bits
- Small hash sizes:
 - Excessive collisions
 - Reduced accuracy
- Large hash sizes:
 - Reduced parameter sharing

Key Observation

Moderate hash configurations provide best performance trade-off.

Embedding Visualization



- t-SNE visualizations show meaningful clusters
- CP-LSH embeddings exhibit improved compactness

Challenges Faced

- Correct implementation of SGNS optimization
- Balancing:
 - Hash collisions
 - Parameter sharing
 - Representation quality
- Hyperparameter sensitivity:
 - Hash tables
 - Hash bits
 - Walk length
- Reproducing large-scale paper results under classroom computational constraints

Limitations

- Experiments conducted on classroom-scale datasets
- Limited GPU/runtime resources
- Full industrial-scale DBLP reproduction was computationally expensive
- Performance sensitive to hashing parameters

- Evaluate on larger industrial datasets
- Explore adaptive hashing strategies
- Combine CP-LSH with Graph Neural Networks
- Improve tail-node representation learning

Conclusion

- Successfully reproduced the CP-LSH framework
- Combined:
 - Structural learning
 - Content-aware parameter sharing
- Experimental findings show:
 - Strong tail-node robustness
 - Competitive overall accuracy
 - Improved scalability potential
- CP-LSH demonstrates advantages for large sparse graphs

Key Takeaways

- DeepWalk remains a strong structural baseline
- CP-LSH improves low-degree node robustness
- LSH enables scalable parameter sharing
- Hybrid structure + content methods are promising
- Scalability becomes increasingly important for large graph datasets

References



Bryan Perozzi et al.

DeepWalk: Online Learning of Social Representations.
KDD 2014.



Aditya Grover and Jure Leskovec.

node2vec: Scalable Feature Learning for Networks.
KDD 2016.



Xiushi Chen et al.

Scalable Graph Representation Learning via Locality-Sensitive Hashing.
CIKM 2022.

Thank You!